

Reading Tracker

System Spec, Data Schema & Session Protocol · v1.0 · May 2026

1 · What This Artifact Is

The Reading Tracker is a dual-purpose system. Privately, it is a structured comprehension and retention tool — addressing two identified weaknesses: reading comprehension and long-term content retention. Publicly, it is documented proof of intellectual engagement and supplemental knowledge growth, published on thepmddev.com as a Readings page and mined for Tier 1 and Tier 2 content strategy output.

The system has three distinct components that work together but remain separate: a JSON data source that grows permanently with every book and chapter entry, a dedicated reading session protocol that governs how comprehension and application work is conducted, and a spaced recall mechanism that enforces structured re-engagement at defined intervals after every chapter is logged.

2 · Three-Component Architecture

Component	File	Purpose	Grows over time?
Data source	PMD_Reading_Data.json	Permanent log of every book, chapter entry, comprehension summary, application notes, and recall evaluations.	Yes — continuously. Never deleted.
Session protocol	Reading session — dedicated conversation	The live coaching interaction. Comprehension test, application ideation, pushback. Separate from project sessions.	No — protocol is stable. Content changes per session.
Recall engine	Embedded in data source — recall_schedule field	Seven-interval spaced recall schedule per chapter. Surfaces due recalls at session open. Pass advances interval. Fail repeats at same interval.	Yes — recall records accumulate permanently.

3 · PMD_Reading_Data.json — Book-Level Schema

The JSON file is structured as a top-level books array. Each book object contains its metadata, ownership status, acquisition flags, phase mapping, and a chapters array. The file is the single source of truth for all reading activity.

Field	Type	Required	Description
schema_version	string	Yes	Schema version — e.g. '1.0'. Increment when fields are added.
generated	string	Yes	ISO 8601 date of last update — e.g. '2026-05-01'.

books[]	array	Yes	Ordered array of all books — queued, in progress, complete, and returned incomplete.
books[].id	string	Yes	Unique book identifier — e.g. 'BOOK-001'. Sequential, never reused.
books[].title	string	Yes	Full book title.
books[].author	string	Yes	Author name(s).
books[].total_chapters	integer	Yes	Total chapter count. Used to calculate progress percentage.
books[].status	string	Yes	One of four states: 'queued' · 'in-progress' · 'complete' · 'parallel'. See Section 4.
books[].progress_pct	number	Yes	Auto-calculated: (chapters logged / total_chapters) × 100. Updated at every chapter entry.
books[].phases_mapped[]	string[]	Yes	PMD phases this book maps to — e.g. ['Phase 1', 'Phase 2', 'Phase 5']. Set at intake. Enables bidirectional phase lookup.
books[].acquisition	object	Yes	Ownership and availability status object. See Section 5.
books[].date_added	string	Yes	Date added to the tracker — e.g. '2026-05-01'.
books[].date_started	string	No	Date first chapter was logged. Null until reading begins.
books[].date_completed	string	No	Date last chapter was logged. Null until all chapters complete.
books[].showcase_candidate	boolean	Yes	true / false — is this book a thepmdev.com Readings page candidate?
books[].showcase_note	string	Yes	One sentence — why it is or is not a Readings page candidate.
books[].content_atoms[]	string[]	Yes — min 1 at complete	Tier 1 content atoms harvested from this book. Populated as chapters are logged. Minimum one entry when book reaches complete status.
books[].chapters[]	array	Yes	Array of chapter log entries. One object per completed chapter. See Section 6.

4 · Four Book States

State	Meaning	Recall active?	Notes
queued	On the reading list. Not yet started. Phase mapping set at intake.	No	Books can be added and removed freely. Removal requires no explanation — circumstances change. Re-adding creates a new entry with original intake date preserved.
in-progress	Currently reading. Chapters being logged.	Yes — per chapter from first entry	Recall clock starts at first chapter log, not at book start. Multiple books may be in-progress simultaneously — see 'parallel' state.
parallel	Actively reading alongside one or more other in-progress books.	Yes — per chapter	Functionally identical to in-progress. Separate state for visibility — the recall dashboard surfaces all parallel books simultaneously so none are accidentally neglected.
complete	All chapters logged. Comprehension and application sessions done.	Yes — ongoing per schedule	Recall continues on the 7-interval schedule for all chapters regardless of book completion. A complete book with outstanding recall intervals is not fully complete.

Returned incomplete books — special handling:

When a borrowed book is returned before all chapters are logged, the book status updates to 'queued' with `acquisition.status` set to 'returned-incomplete'. The `progress_pct` field shows exactly how far was reached — e.g. 64%. All logged chapter entries are retained permanently. A `re_acquire` flag is set to true and the book surfaces in the re-acquisition queue. When re-acquired, a new acquisition object is appended to the acquisition history and reading resumes from the next unlogged chapter. This is distinct from a book that is borrowed, fully completed, and returned — those reach 'complete' status before the physical book is returned and are flagged `acquisition.status` 'returned-complete'.

5 · Acquisition Object Schema

Every book carries an acquisition object that tracks ownership, availability, return obligations, and re-acquisition needs. This is the operational layer that makes the reading list reflect real-world constraints.

Field	Type	Required	Description
<code>acquisition.type</code>	string	Yes	One of four types: 'owned' · 'borrowed-personal' · 'borrowed-library' · 'pending-acquisition'. See type definitions below.
<code>acquisition.status</code>	string	Yes	Current status: 'in-possession' · 'returned-complete' · 'returned-incomplete' · 'not-yet-acquired'.
<code>acquisition.re_acquire</code>	boolean	Yes	true when book needs to be re-acquired to complete remaining chapters. Auto-set true when status becomes 'returned-incomplete'.

acquisition.owner_name	string	Conditional	Required when type is 'borrowed-personal'. Name of the person the book belongs to.
acquisition.library_name	string	Conditional	Required when type is 'borrowed-library'. Name of the library.
acquisition.return_by	string	Conditional	Required when type is 'borrowed-personal' or 'borrowed-library'. ISO 8601 return deadline — e.g. '2026-06-01'. Surfaces as a priority flag when deadline is within 7 days.
acquisition.purchase_notes	string	No	Optional note for type 'pending-acquisition' — e.g. 'available on Amazon', 'check library first', 'wait for paperback edition'.
acquisition.history[]	array	No	Array of prior acquisition objects appended when a book changes hands — e.g. borrowed, returned, then re-acquired. Preserves the full ownership timeline.

Type	Meaning	Return obligation?	Return deadline field?
owned	Personal library — book is yours permanently.	No	No
borrowed-personal	Borrowed from a named individual — friend, colleague, family.	Yes	Yes — required
borrowed-library	Borrowed from a public or institutional library.	Yes	Yes — required
pending-acquisition	Book is queued but not yet physically available — needs to be purchased, requested, or located.	No	No

6 · Chapter Entry Schema

Each chapter entry is the output of a completed reading session — not the session itself. The coaching conversation happens live. What is logged is what survived the session: the approved comprehension summary, the application ideas that held up under pushback, and the recall schedule that begins immediately.

Field	Type	Required	Description
chapter_number	integer	Yes	Chapter number within the book.
chapter_title	string	Yes	Chapter title or descriptive label if untitled.
date_read	string	Yes	Date reading was completed and session was held — ISO 8601.

comprehension_summary	string	Yes	The approved written summary produced during the reading session. Coach-reviewed. Demonstrates understanding without re-stating the chapter verbatim. Target length: 100–200 words.
key_concepts[]	string[]	Yes — min 2	Array of discrete concepts, frameworks, or principles from this chapter. Used as recall prompts at spaced intervals.
application_ideas[]	object[]	Yes — min 1	Array of application idea objects. Each has: idea (string), project_ref (PMD project id or 'future'), phase_ref (PMD phase), and coach_verdict ('approved' · 'needs-development' · 'rejected-rationale'). Only coach-approved ideas are logged.
phase_connections[]	string[]	Yes — min 1	Which PMD phases this chapter's content most directly applies to. Bidirectional — used to surface this chapter when a matching phase is active in a project session.
content_atoms[]	string[]	Yes — min 1	Tier 1 content atoms extracted from this chapter for the content strategy pipeline.
recall_schedule{}	object	Yes	Spaced recall schedule object. See Section 7.
coach_session_notes	string	Yes	One to three sentences of coach feedback from the reading session — what the coach challenged, what was strengthened, what to watch for in the next chapter.

7 · Spaced Recall Schema — Seven-Interval Schedule

The recall schedule begins at chapter completion date, not book completion. A chapter logged today starts its recall clock today — even if the book is still in progress and chapters ahead have not been read. This addresses the real retention problem: it may be weeks since the first chapter was covered by the time the last chapter is logged.

Pass at interval X → advance to interval Y. Fail at interval X → coach provides targeted content reinforcement, then recall is retested at interval X again. The interval does not advance until the recall passes. This is intentionally demanding given the stated goal of ingesting high volume content in compressed time.

Interval	Days from chapter log date	Pass outcome	Fail outcome
R1	Day 3	Advance to R2	Coach reinforcement + retest at Day 3 from fail date

R2	Day 10	Advance to R3	Coach reinforcement + retest at Day 10 from fail date
R3	Day 20	Advance to R4	Coach reinforcement + retest at Day 20 from fail date
R4	Day 30	Advance to R5	Coach reinforcement + retest at Day 30 from fail date
R5	Day 45	Advance to R6	Coach reinforcement + retest at Day 45 from fail date
R6	Day 60	Advance to R7	Coach reinforcement + retest at Day 60 from fail date
R7	Day 90	Recall complete — chapter archived as retained	Coach reinforcement + retest at Day 90 from fail date

Recall schedule object — field definitions:

Field	Type	Description
chapter_log_date	string	ISO 8601 date the chapter was logged. This is the anchor date for the entire schedule.
current_interval	string	Active recall interval — 'R1' through 'R7', or 'complete'.
next_due_date	string	ISO 8601 date when the next recall is due. Surfaced at session open if date has passed.
intervals[]	object[]	Array of completed interval records. Each has: interval_id, due_date, completed_date, result ('pass' or 'fail'), and coach_reinforcement_note (populated on fail).
fully_retained	boolean	true only when R7 is passed. A book is not considered fully processed until all its chapters reach fully_retained: true.

8 · Reading Session Protocol

Reading sessions are dedicated conversations — separate from PMD project sessions. A reading session is not a note-taking exercise. It is a coached comprehension and application workout. The session produces the chapter log entry. The log entry is the output. The conversation is the process.

8.1 · Session Open — Standard Sequence

Step	Action	Who
------	--------	-----

1	Recall surface. Coach checks PMD_Reading_Data.json for any chapters with next_due_date on or before today. If recalls are due, they run before any new chapter work. No new chapter is logged until due recalls are handled.	Coach
2	Return deadline check. Coach flags any book with acquisition.return_by within 7 days. Informs prioritisation of the session's reading focus.	Coach
3	Session scope confirmation. User states which book and chapter the session covers. Coach confirms chapter against the books[] array — ensures continuity.	User + Coach

8.2 · Comprehension Phase

Step	Action	Coach behaviour
1	User writes a summary of the chapter without referring back to the text. No notes. Freeform. Target: 100–200 words.	Silent until summary is submitted.
2	Coach evaluates the summary against two criteria: accuracy (did the user capture what the chapter actually argues?) and depth (did the user go beyond surface re-stating to capture the underlying principle?).	Specific feedback on both criteria. Not generic encouragement.
3	User revises the summary based on coach feedback until it meets both criteria.	Pushback continues until the summary is genuinely accurate and deep. The approved summary is logged in comprehension_summary.
4	User identifies 2–4 key concepts from the chapter.	Coach challenges vague or overlapping concepts. These become the recall prompts.

8.3 · Application Phase

Step	Action	Coach behaviour
1	Coach prompts: 'How does this chapter's core principle apply to any past, current, or future PMD project — or more broadly to product work?' User responds with at least one specific application idea.	Asks the prompt. Does not suggest ideas first.
2	For each application idea, coach pushes back with one challenge: 'How specifically? What would change in the project if this principle were applied? What would the output look like differently?'	Precision over encouragement. Vague application ideas are returned, not accepted.

3	Ideas that survive pushback are logged in <code>application_ideas[]</code> with <code>coach_verdict</code> 'approved'. Ideas that need more development are logged as 'needs-development' with a note. Ideas that do not hold up are logged as 'rejected' with rationale.	Every idea gets a verdict — nothing is left unresolved.
---	---	---

8.4 · Session Close

Step	Action	Who
1	Coach extracts Tier 1 content atoms from the session. States them explicitly. User confirms or adds. Logged in <code>content_atoms[]</code> .	Coach + User
2	Coach writes one to three sentences of session feedback — what was strong, what to sharpen, what to watch for in the next chapter. Logged in <code>coach_session_notes</code> .	Coach
3	Chapter entry is committed to <code>PMD_Reading_Data.json</code> . Recall schedule R1 is set with <code>next_due_date</code> = session date + 3 days.	Coach confirms entry structure.

9 · Bidirectional Phase Mapping

Phase mapping works in two directions. At book intake, phases are assigned to the book — this book maps to Phase 2 Research and Phase 5 PRD. At the chapter level, phases are assigned per chapter — this specific chapter is most relevant to Phase 3 Strategy. The two layers together enable two distinct lookup modes:

Lookup direction	Query	Source fields	Use case
Book → Phase	'What books have I read that are relevant to Phase 5?'	<code>books[].phases_mapped[]</code>	Phase kickoff reading list. Before starting a new phase, surface all logged books mapped to it.
Chapter → Phase	'Which specific chapters are most relevant to what I am working on in Phase 3 right now?'	<code>chapters[].phase_connections[]</code>	Active project reference. Mid-phase, surface the specific chapters most applicable to the current decision.
Phase → Book + Chapter	'I am in Phase 7 Build — what reading applies?'	Both fields combined	Full reading reference for any active phase. Returns both book-level and chapter-level matches.

10 · Output Files — Naming Convention

File	Format	Purpose
PMD_Reading_Data.json	JSON	Permanent data source. Lives at repo root. Never deleted. All books, chapters, recall records.
thepmdev.com/readings	Web page	Public Readings page. Showcase-candidate books only. Chapter summaries + application ideas published. P0 v2 build.
Tier 1 posts from content_atoms[]	Social / blog	Build Log posts extracted from chapter content atoms at session close.
Tier 2 pillars from aggregated chapters	Medium / Substack	Insight Pillar posts aggregated from a book's complete chapter content atoms.

11 · Locked Decisions

Decision	Detail
Log and session are separate	The reading session is a live coaching conversation. The log captures what survived it. Over-engineering the log or under-delivering the session are both failure modes. Kept strictly separate.
Recall starts at chapter completion	Not at book completion. A chapter logged today starts its recall clock today. Addresses the real problem: weeks may pass between chapter 1 and the final chapter.
Seven-interval schedule	3 → 10 → 20 → 30 → 45 → 60 → 90 days. Intensive by design for high-volume intake. Can be dialled back at a later session if proving excessive. Starting intensive is cheaper than trying to rebuild retention after gaps form.
Fail repeats at same interval	A failed recall is retested at the same interval from the fail date — not advanced. Advancing on a fail defeats the entire purpose of the schedule.
Bidirectional phase mapping	Book-level mapping at intake. Chapter-level mapping at chapter log. Both directions enable distinct lookup modes — reading list by phase, and active reference during project work.
Four acquisition types	owned · borrowed-personal · borrowed-library · pending-acquisition. Return deadlines surface as priority flags within 7 days. Real operational constraints reflected in the schema.
Returned-incomplete retention	All logged chapters from a returned-incomplete book are retained permanently. Progress percentage shown. Re-acquisition queued. Distinct from returned-complete.
Four book states including parallel	queued · in-progress · parallel · complete. Parallel state is visible so multiple active books surface simultaneously on the recall dashboard — none are neglected.
Dedicated reading sessions	Separate from project sessions. Global applicability across all projects — not tied to real-time individual project decisions. Cross-reference via phase_connections[] handles intersections without merging session types.

P() helper — all table cells

Per locked PDF pattern. All table cell content wrapped in Paragraph. VALIGN TOP. Never plain strings.

12 · Example JSON Seed — First Book Entry

Abbreviated example showing the full structural skeleton for one book with one logged chapter. Replace placeholder values at first use.

```
{
  "schema_version": "1.0",
  "generated": "2026-05-01",
  "books": [
    {
      "id": "BOOK-001",
      "title": "[Book Title]",
      "author": "[Author Name]",
      "total_chapters": 12,
      "status": "in-progress",
      "progress_pct": 8,
      "phases_mapped": ["Phase 2", "Phase 3"],
      "acquisition": {
        "type": "owned",
        "status": "in-possession",
        "re_acquire": false
      },
      "date_added": "2026-05-01",
      "date_started": "2026-05-01",
      "date_completed": null,
      "showcase_candidate": true,
      "showcase_note": "[One sentence – why or why not a Readings page candidate]",
      "content_atoms": [],
      "chapters": [
        {
          "chapter_number": 1,
          "chapter_title": "[Chapter Title]",
          "date_read": "2026-05-01",
          "comprehension_summary": "[100-200 word approved summary]",
          "key_concepts": ["[Concept 1]", "[Concept 2]"],
          "application_ideas": [
            {
              "idea": "[Application idea description]",
              "project_ref": "P1",
              "phase_ref": "Phase 3",
              "coach_verdict": "approved"
            }
          ],
          "phase_connections": ["Phase 2", "Phase 3"],
          "content_atoms": ["[Tier 1 atom from this chapter]"],
          "recall_schedule": {
            "chapter_log_date": "2026-05-01",
            "current_interval": "R1",
            "next_due_date": "2026-05-04",
```

```
"intervals": [],
"fully_retained": false
},
"coach_session_notes": "[1-3 sentences of coach feedback from the session]"
}
]
}
]
}
```

Version History

Version	Key change	Date
v1.0	Initial spec — three-component architecture, full book and chapter schema, four acquisition types, four book states including parallel, seven-interval spaced recall with pass/fail mechanics, dedicated reading session protocol with comprehension and application phases, bidirectional phase mapping, returned-incomplete handling, content pipeline integration.	May 2026